河南大学 2022 届本科毕业论文

基于 GO 语言的 Kademlia 网络协议的设计与开发

Design and Development of Kademlia Network Protocol Based on GO Language

论文作者姓名：李嘉恒

作 者 学 号：1825010302

所 在 学 院：软件学院

所 学 专 业：网络工程

导 师 姓 名：谢苑

导 师 职 称：讲师

2022 年 4 月 25 日

河南大学 2022 届本科生毕业论文（设计、创作）任务书

论文（设计、创作）题目	基于 GO 语言的 Kademlia 网络协议的设计与开发		
所在学院	软件学院	专　业	网络工程
学生姓名	李嘉恒	学　号	1825010302

一、毕业论文（设计、创作）要求

1．设计内容

利用大学四年所学知识为基础，搜索分布式网络通讯原理相关资料，使用 Go 语言开发 Kademlia 网络协议。本设计将使用当今的结构化网络中最流行的 Kademlia 协议，拓扑结构使用树形结构，RPC 使用公共开源协议 PING，FIND_NODE，GET_PEERS，ANOUNCE_PEERS。需要完成用户级功能：加入网络，退出网络，查找结点，查找资源，下载资源五个功能。

2．基本要求

1.学习 Kademlia 协议与模型。

2.完成 Kademlia 协议模型设计。

3.学习 Go 语言开发。

4.代码实现计划的模型与功能。

3．要求提交的成果

（1）毕业设计源程序

（2）论文

二、毕业论文（设计、创作）进度安排

2021 年 10 月至 2021 年 11 月完成并提交开题报告；

2021 年 11 月至 2022 年 1 月底完成毕业设计的代码实现并填充好各部分功能，提交中期检查报告；

2022 年 2 月至 2022 年 3 月中旬完成毕业论文初稿；

2022 年 3 月中旬到 2022 年 4 月完成论文修改和查重，达到答辩要求。

三、需收集的资料和指导性参考文献

[1] 黄煜东. 基于 rust 编程语言的 i2p 匿名通信设计[D].华中师范大学,2018.

[2] Maymounkov P, Mazieres D. Kademlia: A peer-to-peer information system based on the xor metric[C]//International Workshop on Peer-to-Peer Systems. Springer, Berlin, Heidelberg, 2002: 53-65.

指导教师签名

2021 年 10 月 12 日

河南大学2022届本科生毕业论文（设计、创作）开题报告

论文（设计、创作）题目	基于GO语言的Kademlia网络协议的设计与开发		
所在学院	软件学院	专　业	网络工程
学生姓名	李嘉恒	学　号	1825010302

一、本课题研究意义

随着全球网络设备完善和网站流量增大，服务器为了满足高并发需求，引入大数据和云计算技术。云计算与大数据技术底层需要使用分布式技术与算法来实现，其中分布式算法性能也决定了云计算系统的性能，进而也影响着云服务器的成本与性能。

本课题将使用当前最流行的，网络性能最好的GO语言对DHT分布式网络中经典的Kademlia网络协议做出实现，使用GoLang中效率更高的协程提高并发量，以达成提高系统效率的目的。

二、国内外有关本课题的研究动态

国内对于DHT网络的研究不多。有部分学者研究设计更新的DHT结构以增加性能，也有许多学者研究DHT网络的爬虫搜索技术。

国外对于DHT研究相比国内较多。近些年国外在DHT的研究主要在对于Chord和Kad系统在安全或效率方面进行改进。

三、本课题研究的基本内容

本课题首先介绍了分布式技术的发展，对当今流行的DHT结构化网络的特点进行了分析。随后列举了结构化网络中最常见的两种设计，分析优劣，认为Kad是目前最优秀的网络。本文根据设计模型和Kad协议规范，依次使用GoLang语言实现二叉树，节点和配置文件等数据结构，路由表，路由算法，通讯协议和并发等设计。

四、本课题拟解决的主要问题

1.解决DHT传输效率低。

2.解决DHT中二叉树结构数据的设计。

五、研究方法

本文将使用论文查阅和代码实现两种方式对 DHT 网络做研究。使用论文查阅了解相关课题研究方向与方法，总结经验做出设计，进行代码实现来完成设计。

六、主要创新点

1.使用GO语言，语言本身带来更加优秀的运行效率。

2.使用Kad规范。

3.采用协程设计多线程。

4.使用二叉树实现路由表代替桶。

5.使用Gomod规范项目结构。

七、主要参考文献

[1] Ma W. Research on how to build new Foreign Trade competitive advantages of Ningbo by using the" Internet+" under the innovation-driven Background[C]//2016 2nd International Conference on Humanities and Social Science Research (ICHSSR 2016). Atlantis Press, 2016: 381-384.

[2] 张玉洁,何明,孟祥武.基于用户需求的内容分发点对点网络系统研究[J].软件学报,2014(1):98-117.

[3] 温安宇. 基于 DHT 的 key-value 分布式存储系统[D].哈尔滨工业大学,2010.

[4] Danezis, G., Lesniewski-Laas, C., Kaashoek, M.F., Anderson, [D] (2005). Sybil-Resistant DHT Routing. In: di Vimercati, S.d.C., Syverson, P., Gollmann, D. (eds) Computer Security – ESORICS 2005. ESORICS 2005. Lecture Notes in Computer Science, vol 3679. Springer, Berlin, Heidelberg.

[5] Crosby S A , Dan S W . An Analysis of BitTorrent's Two Kademlia-Based DHTs. 2007.

[6] Stoica I, Morris R, Liben-Nowell D, et al. Chord: a scalable peer-to-peer lookup protocol for internet applications[J]. IEEE/ACM Transactions on networking, 2003, 11(1): 17-32.

[7] Douceur J R. The sybil attack[C]//International workshop on peer-to-peer systems. Springer, Berlin, Heidelberg, 2002: 251-260

[8] Maymounkov P, Mazieres D. Kademlia: A peer-to-peer information system based on the xor metric[C]//International Workshop on Peer-to-Peer Systems. Springer, Berlin, Heidelberg, 2002: 53-65.

[9] 黄煜东. 基于 rust 编程语言的 i2p 匿名通信设计[D].华中师范大学,2018.

具体时间及写作进度安排

起止日期	主要工作内容
2021.9.20-2021.11.14	与导师交流，查找资料，确定毕业设计选题，撰写开题报告。
2021.11.15-2021.11.20	修改完善开题报告，提交给导师审核。
2021.11.21-2021.11.30	答辩委员会（组）完成开题审查。
2021.12.01-2022.03.15	根据所选课题进行毕业设计，填写中期检查表提交给导师，导师负责审查，答辩委员会（组）审核。
2022.03.16-2022.04.20	根据导师的建议和指导，撰写毕业论文，完成论文查重。
2022.04.21-2022.04.30	论文评阅和答辩。
2022.05.01-2022.05.10	根据答辩建议修改毕业论文。

指导教师对开题报告的意见

此项目选用比较冷门的课题，但是课题内容具有深刻意义，其中不论是使用的编程语言还是系统都难度较大，具有挑战性。此项目中的 DHT 结构对于云计算等领域也有指导意义。

同意开题。

指导教师签名： 谢苑 **2021 年 11 月 30 日**

系（室）对本课题开题的意见

此项目选用比较冷门的课题，但是课题内容具有深刻意义，其中不论是使用的编程语言还是系统都难度较大，具有挑战性。

同意开题。

负责人签名： 韩志杰

2021 年 11 月 30 日

河南大学 2022 届本科生毕业论文（设计、创作）

中期进展情况检查表

论文（设计、创作）题目	基于 GO 语言的 Kademlia 网络协议的设计与开发		
所在学院	软件学院	**专　业**	网络工程
学生姓名	李嘉恒	**学　号**	1825010302

一、毕业论文（设计、创作）进展情况

系统已经完成设计。项目目前已完成大多数工作，已完成路由固化，硬盘存储，网络端口监听，路由表与路由表生成，二叉树与路由表转换，远程调用命令协议，以及系统中呼吸频率与收包频率等系统配置模块。剩余较难的协程管理模块尚未完成。

二、毕业论文（设计、创作）存在问题及解决方案

存在问题：

1.协程管理与线程本质不同，无法采用原先设计直接把桶作为线程。

2.由于需要引用编码等工具包，需要模块管理，而有些模块版本过老，无法支持高版本的 GO 语言。

解决方案：

1.进一步查找资料学习 GO 语言协程相关知识。

2.自己编写相关工具库。

三、指导教师对学生论文（设计、创作）进展等方面的评价

该生的程序设计进展比较顺利，大多数系统代码已经完成。已完成路由固化，硬盘存储，网络端口监听，路由表与路由表生成，二叉树与路由表转换，远程调用命令协议，以及系统中呼吸频率与收包频率等系统配置模块。但是多协程内容遇到困难，还需要查询资料进一步完善。系统中也存在一些细节问题，希望进一步改正。

该生总体进展复合预期，可以在规定的时间内完成论文的攥写，修改工作。

指导教师签字： 谭苑 **2022 年 3 月 15 日**

系（室）对本课题中期检查的意见

该生的程序设计进展比较顺利，大多数系统代码已经完成，但是项目遇到了一些困难，还需要查询资料进一步完善，系统中也存在一些细节问题，希望进一步改正。

该生总体进展复合预期，可以在规定的时间内完成论文的攥写，修改工作。

负责人签名： 韩志杰 **2022 年 3 月 15 日**

河南大学 2022 届毕业论文（设计、创作）指导教师评价表

<table>
<tr><td colspan="2">论文（设计、创作）题目</td><td colspan="4">基于 GO 语言的 Kademlia 网络协议的设计与开发</td></tr>
<tr><td colspan="2">所在学院</td><td>软件学院</td><td>专　业</td><td colspan="2">网络工程</td></tr>
<tr><td colspan="2">学生姓名</td><td>李嘉恒</td><td>学　号</td><td colspan="2">1825010302</td></tr>
<tr><td colspan="2">指导教师</td><td>谢苑</td><td>指导教师职称</td><td colspan="2">讲师</td></tr>
<tr><td rowspan="9">指导教师评分</td><td rowspan="2">序号</td><td rowspan="2">评分项目</td><td colspan="2">评分参考</td><td>得分</td></tr>
<tr><td>评阅指标（优秀标准）</td><td>满分（100 分）</td><td></td></tr>
<tr><td>1</td><td>选题质量</td><td>符合专业培养目标；有实际意义和推广价值。</td><td>20</td><td>18</td></tr>
<tr><td>2</td><td>文献资料 应用能力</td><td>能独立查阅文献，具有收集、加工各种信息及获取新知识的能力。</td><td>10</td><td>8</td></tr>
<tr><td>3</td><td>调查研究能力</td><td>能准确理解课题任务；研究方案设计合理；能独立从事调查研究；能综合运用所学知识发现与解决实际问题。</td><td>30</td><td>27</td></tr>
<tr><td>4</td><td>论文（设计、创作质量</td><td>格式、图表（或图纸）规范，符合要求；结构严谨，逻辑性强；内容翔实，表达准确流畅；学术价值或实用价值高。</td><td>20</td><td>17</td></tr>
<tr><td>5</td><td>创新能力</td><td>观点独到，方法新颖，角度新颖。</td><td>10</td><td>8</td></tr>
<tr><td>6</td><td>工作量及态度</td><td>工作量饱满；能圆满完成任务书规定的各项工作。</td><td>10</td><td>8</td></tr>
<tr><td colspan="3">总得分</td><td colspan="2">86</td></tr>
<tr><td rowspan="3">指导教师评定意见</td><td colspan="5">参照上述评价标准及学生论文（设计、创作）完成情况，做出具体评价：</td></tr>
<tr><td colspan="5">该生能比较全面地完成毕业设计的任务，方案合理。该论文通过从国内外现状入手，论文综合运用了大量计算机网络知识解决了现有问题。该系统遵守国际协议规范的 PRC，使用较新技术完成系统，同时对系统做出创新和改进。该生分析问题与解决问题能力较强，能对所学知识灵活运用。该生能够运用本学科的常规研究方法与分析方法，运用相关开发技术和软件，进行系统设计与编写。
符合毕业要求，同意答辩。</td></tr>
<tr><td colspan="5">指导教师签名：谢苑　　2022 年 4 月 25 日</td></tr>
</table>

河南大学 2022 届毕业论文（设计、创作）评阅教师评价表

<table>
<tr><td colspan="2">论文（设计、创作）题目</td><td colspan="4">基于 GO 语言的 Kademlia 网络协议的设计与开发</td></tr>
<tr><td colspan="2">所在学院</td><td>软件学院</td><td>专　业</td><td colspan="2">网络工程</td></tr>
<tr><td colspan="2">学生姓名</td><td>李嘉恒</td><td>学　号</td><td colspan="2">1825010302</td></tr>
<tr><td colspan="2">评阅教师</td><td>谢苑</td><td>评阅教师职称</td><td colspan="2">讲师</td></tr>
<tr><td rowspan="8">评阅教师评分</td><td rowspan="2">序号</td><td rowspan="2">评分项目</td><td colspan="2">评分参考</td><td>得分</td></tr>
<tr><td>评阅指标（优秀标准）</td><td>满分
（100 分）</td><td></td></tr>
<tr><td>1</td><td>选题质量</td><td>符合专业培养目标：有实际意义和推广价值。</td><td>20</td><td>18</td></tr>
<tr><td>2</td><td>文献资料应用能力</td><td>能独立查看文献，具有收集，加工各种信息及获取新知识的能力。</td><td>10</td><td>8</td></tr>
<tr><td>3</td><td>调查研究能力</td><td>能准确理解课题任务：研究方案设计合理:能独立从事调查研究，能综合运用所学知识发现与解决实际问题。</td><td>30</td><td>27</td></tr>
<tr><td>4</td><td>论文（设计、创作）质量</td><td>格式，图表（或图纸）规范符合要求：结构严谨，逻辑性强 内容翔实，表达准确流畅：学术价值或使用价值高。</td><td>30</td><td>27</td></tr>
<tr><td>5</td><td>创新能力</td><td>观点独到，方法新颖，角度新颖。</td><td>10</td><td>8</td></tr>
<tr><td colspan="3">总得分</td><td colspan="2">88</td></tr>
<tr><td rowspan="3">评阅教师评定意见</td><td colspan="5">参照上述评价标准及论文（设计、创作）内容，做出具体评价：</td></tr>
<tr><td colspan="5">该论文内容完整，整体结构符合论文的写作规范。该论文研究的分布式网络对学习发展有促进作用，增加学习者互相交流的能力，具有现实意义。毕业设计系统和论文题目及内容相符，能够将自己在学习所学的计算机网络理论知识运用到程序中。论文层次结构安排科学，逻辑关系较为清晰，论文语言表达基本通畅，论文格式基本符合学院论文规范要求。
总体上达到毕业要求，同意答辩。</td></tr>
<tr><td colspan="5">评阅教师签名：刘颜红　　2022 年 4 月 29 日</td></tr>
</table>

1、河南大学2022届本科生毕业论文（设计、创作）答辩成绩表

<table>
<tr><td colspan="2">论文（设计、创作）题目</td><td colspan="5">基于GO语言的Kademlia网络协议的设计与开发</td></tr>
<tr><td colspan="2">所在学院</td><td colspan="2">软件学院</td><td>专　业</td><td colspan="2">网络工程</td></tr>
<tr><td colspan="2">学生姓名</td><td colspan="2">李嘉恒</td><td>学　号</td><td colspan="2">1825010302</td></tr>
<tr><td rowspan="3">答辩委员会（组）评分及评定结论</td><td>评分项目及分值</td><td colspan="6">答辩委员会（组）专家评分</td></tr>
<tr><td></td><td colspan="2">答　辩　情　况</td><td colspan="2">论　文　质　量</td><td rowspan="2">合计
（100分）</td></tr>
<tr><td></td><td>内容表达情况
（15分）</td><td>回答问题情况
（25分）</td><td>规范要求与文字表达
（20分）</td><td>论文（设计、创作）质量和创新意识
（40分）</td></tr>
<tr><td></td><td>得分</td><td>13</td><td>23</td><td>28</td><td>34</td><td>88</td></tr>
<tr><td></td><td>答辩委员会（组）评定结论</td><td colspan="5">该生语言叙述流畅，论文研究DHT网络结构，选题新颖，具有一定的理论价值和现实指导意义。该生在论文写作过程中态度认真，广泛查阅文献和相关资料，比较完整地实现了论文初期所提出的要求和内容，按时完成了写作任务。论文格式正确完整，内容全面充分。
总体达到了毕业答辩的要求。通过答辩。
答辩委员会（组）签字：
2022年4月30日</td></tr>
<tr><td colspan="7">毕业论文（设计、创作）答辩成绩：　88　分
答辩委员会（组）负责人签字：
2022年4月30日</td></tr>
</table>

2、备注：

3、1.论文的质量评定，应包括对论文的语言表达、结构层次、逻辑性理论分析、设计计算、数据处理、分析和概括能力及在论文中是否有新的见解或创造性成果等做出评价。从论文来看学生掌握本专业基础理论和基本技能的程度。

4、2.评分由答辩委员会（组）专家（原则上5或7人一组）操作，根据学生答辩情况，答辩委员会专家及时给出每个学生的成绩。内部对评分如有争议时，应当场进行表决。

5、3.**答辩委员会（组）评定结论应说明是否通过答辩。**

6、4.各院亦可根据本专业的不同情况，制定相应的具有自己特色的评价表格。

7、5.以上表格所有签字部分必须手签。

河南大学2022届本科生毕业论文（设计、创作）

答辩纪要

论文(设计、创作）题目	基于GO语言的Kademlia网络协议的设计与开发		
答辩人姓名	李嘉恒	**指导教师**	谢苑
答辩时间	具体答辩时间	**答辩地点**	具体答辩地点
答辩委员会(组)负责人	韩志杰	**答辩委员会（组）成员**	张新红、渠慎明、王秋雨、段玉春、李涵

答辩中提出的问题及回答的简要情况记录：

问：你的功能能不能用比较通俗的语言给我们说一下。

答：通俗的来讲，我的系统准确来说是一个协议。通俗来讲就是像我们常见的种子下载，P2P下载，种子下载是没有服务器的。我的系统这些功能主要就是为了完成在没有服务器的情况下，怎么进行节点之间的通讯。在我的这个系统里面他主要是做定位。这个系统里的其他RPC都是为了辅助GET_PEERS和FIND_NODE。去PING其它节点帮助更新整个路由表使用。

问：你现在做的这个作品目前阶段，你认为是一个从技术层面上来说是一个比较好的还是说一个在本科阶段已经属于在你的能力范围内比较先进的一个技术手段。

答：我这个结构目前已经被使用在种子下载，电驴下载，比特币中。但是这个系统所在的是分布式领域，分布式领域在学校是几乎见不到的。学习时比较困难，很少有资源和书籍，更多是通过一个个分散的博客，每个人都写一点自己的理解，我把他们的博客串联起来。有时候还需要查看原始的英文论文去学习。我无法判断这个内容是不是本科阶段能否完成，但是对于整个分布式网络来说，我的内容还是比较基础的东西。就以比特币举例，他的网络层其实使用了两个技术，DHT和PEX，因为咱们使用的网络都存在NAT，PEX是做NAT穿透用的。但是很遗憾我还没有研究到这个程度

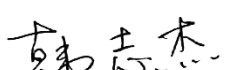
记录人签名： 谢苑

2022 年 4 月 30 日

河南大学2022届本科生毕业论文（设计、创作）

综合成绩表

<table>
<tr><td>论文（设计、创作）题目</td><td colspan="5">基于GO语言的Kademlia网络协议的设计与开发</td></tr>
<tr><td>所在学院</td><td colspan="2">软件学院</td><td>专　业</td><td colspan="2">网络工程</td></tr>
<tr><td>学生姓名</td><td colspan="2">李嘉恒</td><td>学　号</td><td colspan="2">1825010302</td></tr>
<tr><td>指导教师评分</td><td>86</td><td>评阅教师评分</td><td>88</td><td>答辩评分</td><td>88</td></tr>
<tr><td>综合成绩</td><td>87</td><td>成绩等级</td><td>良好</td><td>是否推优</td><td>否</td></tr>
<tr><td>所在系（室）意见</td><td colspan="5">该生的论文具有很强的理论价值，能够将自己在学习所学的计算机网络和数据结构理论知识运用到实践中。设计中使用大量二叉树等数据结构和网络通讯知识，知识运用灵活。该论文研究的DHT网络对学习发展也有促进作用，增加学习者互相交流的能力，具有现实意义。论文选题具有较强的新颖性。
该论文思路清晰、内容充实、观点明确、论据充分、论证严格，整篇论文的逻辑性强，层次清晰，结构合理，文笔流畅，完全符合论文的标准和规范。

负责人签名：  2022 年 5 月 4 日</td></tr>
<tr><td>学院意见（仅推优论文填写）</td><td colspan="5">负责人签名（盖章）：

2022 年 5 月 4 日</td></tr>
</table>

8、备注：

9、毕业论文（设计、创作）综合成绩以百分制计，构成方式为：指导教师评分（30%）+评阅教师评分（20%）+答辩成绩（50%），各部分评分可保留两位小数，最终综合成绩需要四舍五入保留整数。

10、毕业论文（设计、创作）综合成绩对应四个等级：优秀：100～90；良好：89～76；中等：75～60；不及格：59～0。

11、河南大学本科生毕业论文（设计、创作）承诺书

论文（设计、创作）题目	基于GO语言的Kademlia网络协议的设计与开发				
姓 名	李嘉恒	**学 号**	1825010302	**专 业**	网络工程
指导教师姓名	谢苑	**职 称**	讲师		
完成时间	2021年9月20日 — 2022年4月30日				

承诺内容：

1、本毕业论文（设计、创作）是学生李嘉恒**在导师**谢苑**的指导下独立完成的，没有抄袭、剽窃他人成果，没有请人代做，若在毕业论文（设计、创作）的各种检查、评比中被发现有以上行为，愿按学校有关规定接受处理，并承担相应的法律责任。**

2、学校有权保留并向上级有关部门送交本毕业论文（设计、创作）的复印件和电子稿件。学校可以将毕业论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

备注：

学生签名： 李嘉恒 **指导教师签名：** 谢苑

2022 年5 月07日 **2022 年5 月07日**

摘　要

随着全球网络设备完善和网站流量增大，服务器为了满足不断增长的高并发和海量数据的需求，引入大数据和云计算技术[1] 。而云计算与大数据技术底层都是将多台服务器组合，服务器与服务器之间的通信与存储需要使用分布式技术与算法来实现。其中分布式算法性能也决定了云计算系统的性能，进而也影响着云服务器的成本与性能。本文将使用当前最流行的，网络性能最好的 GO 语言对 DHT 分布式网络中经典的 Kademlia 网络协议做出实现，使用 GoLang 中效率更高的协程提高并发量，以达成提高系统效率的目的。

本文首先介绍了分布式网络的相关技术。接着本文介绍了改进的 Kademlia 的系统模型设计。然后本文根据设计模型和 Kad 协议规范，使用 GoLang 语言，依次实现二叉树，节点和配置文件等数据结构，实现路由表，路由算法，通讯协议和并发等功能模块，实现加入网络，搜索节点，搜索资源，下载资源，退出网络五个用户级功能。

关键词： DHT；Kademlia；P2P；分布式；GoLang

ABSTRACT

With the improvement of global network equipment and the increase of website traffic, servers are introducing big data and cloud computing technology techniques in order to meet the growing demand of high concurrency and massive data[1] . The cloud computing and big data technology underlying both combine multiple servers, and the communication and storage between servers need to be implemented using distributed technologies and algorithms. The performance of the distributed algorithms also determines the performance of the cloud computing system, which in turn also affects the cost and performance of the cloud server. In this thesis, we use the most popular and best network performance GO language to implement the classic Kademlia network protocol in DHT distributed network, and use the more efficient concurrency in GoLang to improve the concurrency to achieve the purpose of improving the system efficiency.

This thesis first introduces the technologies related to distributed networks. Then this thesis presents the system model design of the improved Kademlia. Then according to the design model and Kad protocol specification, by using Go language, this thesis implements the data structures such as binary tree, nodes and configuration files in order to realize the functional modules such as routing table, routing algorithm, communication protocol and concurrency, and five user-level functions such as join the network, search nodes, search resources, download resources and exit the network.

Keywords: DHT; Kademlia; P2P; distributed; GoLang

目 录

摘 要 I
ABSTRACT II
第 1 章 绪 论 4
1.1 课题来源 4
1.2 研究背景 4
1.3 国内外在该方向的研究现状及分析 5
1.3.1 国内现状 5
1.3.2 国外现状 5
1.4 设计目标 6
1.5 使用的开发平台 6
1.5.1 Linux&Windows 6
1.5.2 GoLand 6
第 2 章 DHT 相关技术分析 7
2.1 Chord 7
2.2 Kademlia 8
2.3 探讨不同 DHT 结构优劣 8
2.3.1 耦合性 8
2.3.2 结构实现难度 9
2.3.3 并发难度 9
2.3.4 安全性 9
第 3 章 系统模型设计 11
3.1 拓扑结构——二叉树 11
3.1.1 预处理 11
3.1.2 散列值映射 11
3.2 距离算法 12

3.3 路由机制.....12
3.4 二叉树拆分.....12
3.5 路由表.....13
3.6 路由表刷新机制.....13
3.7 并发请求.....14
3.8 协议消息.....14
3.9 流程控制.....14
3.9.1 节点查询.....15
3.9.2 节点定位.....15
3.9.3 保存资源.....16
3.9.4 加入网络.....16
3.9.5 退出网络.....16
第 4 章 系统代码开发.....17
4.1 实现原则.....17
4.2 系统开发环境配置.....18
4.3 数据结构 Data Structure.....18
4.3.1 Type DHT.....18
4.3.2 Type Config.....20
4.4 散列值映射实现.....21
4.5 路由表实现.....21
4.6 远程过程调用协议 RPC 实现.....21
4.6.1 PING 实现.....21
4.6.2 FIND_NODE 实现.....22
4.6.3 GET_PEERS 实现.....22
4.6.4 ANNOUNCE_PEER 实现.....23
4.7 用户级功能.....23
4.7.1 加入网络实现.....23
4.7.2 退出网络实现.....24

4.7.3 查找节点实现 24
4.7.4 查找资源实现 25
4.7.5 保存资源实现 25
4.8 系统主函数 25
4.9 协程 Goruntine 27
4.10 存储 27
第 5 章 测 试 28
5.1 测试查询资源 28
5.2 测试下载资源 29
结 语 30
参考文献 31
致 谢 32

第 1 章　绪　论

分布式网络，又称 P2P 对等式网络，是一种点对点技术。它曾用于减少资源服务器压力，文件分享。现如今，云计算技术底层也需要 P2P 技术来满足需求，P2P 也是区块链技术的网络底层。近年来，GoLang 语言在全球流行，成为多家互联网公司的使用语言。该语言有着诸多性能优势。因此本文希望使用 GoLang 语言重写 DHT，使用 GoLang 中效率更高的协程提高并发量，以达成提高系统效率的目的。

1.1 课题来源

该课题为自选课题。

1.2 研究背景

P2P 技术起初是为了减轻服务器压力而诞生。最早的 P2P 技术采用中央服务器的模式。节点连接到中央服务器，节点向服务器查询文件的位置后，与节点建立通讯。这种设计并不能完全解决问题，服务器本身还是成为了整个系统的性能瓶颈。

为了解决上述问题，随后的 P2P 网络采用广播的设计模式。当节点要查询文件时会按照规则进行遍历。规则要求：当节点需要获取资源时，向路由表中的所有在线节点进行发出资源获取请求；节点接受请求，如果有资源则返回资源；如果不存在，则重复上一步向已知节点发出广播搜索。直到获取到资源为止。

这类技术存在巨大缺陷，当节点查询的资源在系统中不存在，或仅在少数节点上保存时，容易引发广播风暴并严重占用网络带宽，也会严重消耗节点的系统资源[2] 。即使在协议层面通过设置数据包生存跳数，限制查询过程递归次数，依然无法彻底解决此弊端。因为其引发的网络瘫痪，被称为 Query Flooding 查询泛洪。如图 1.1 所示。现在 Query Flooding 也被视为一种网络攻击。

此后的 P2P 技术为了解决上述问题，开始引入分布式哈希表 DHT，为网络设计拓扑结构，甚至 RSA 身份认证等技术。引入 DHT 是为了给系统中的节点一个查询的方向，这样节点查询资源时就不需要发送广播。起初 DHT 在种子，电驴和磁力链接中广泛应用。在区块链技术中，DHT 与 PEX（peer exchange）节点交换技术结合被设计作为区块

链底层链路。在云计算中，多种 DHT 协议被设计作为服务器之间底层网络，并用于通讯和存储。

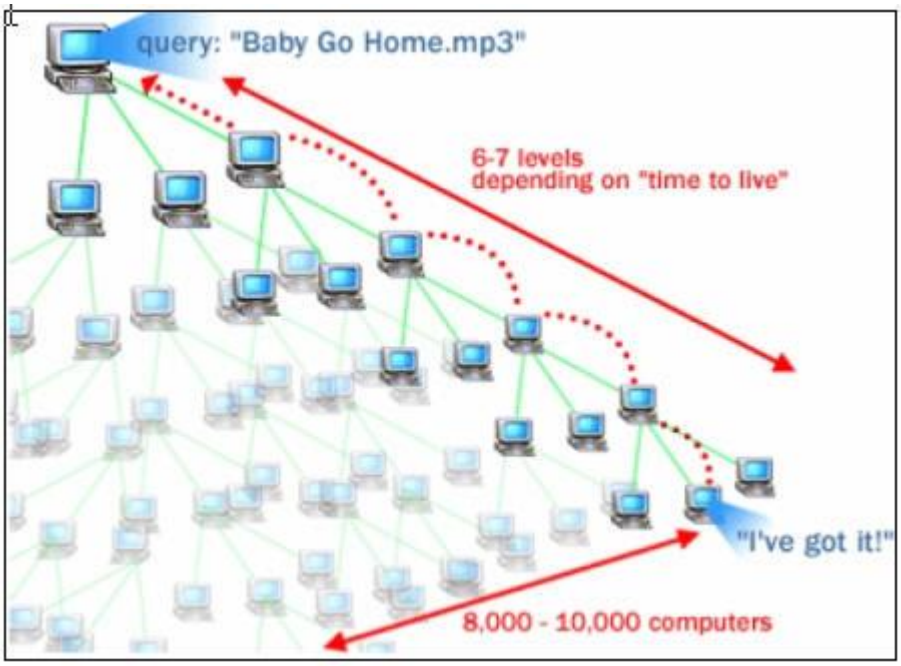


图 1.1 Query Flooding

1.3 国内外在该方向的研究现状及分析

国内外近年来对 DHT 的研究都较为冷门，仅有少数研究者发表论文。

1.3.1 国内现状

国内对于 DHT 网络的研究不多。如何充分利用这些终端资源，在高动态的网络环境中构建大规模、高可扩展性、高可靠性、高性能的存储系统，成为近年来的研究热点之一[3] 。较多学者在研究 DHT 网络的爬虫搜索技术。

1.3.2 国外现状

国外对于 DHT 研究稍比国内活跃。近些年国外在 DHT 的研究主要对 Chord 或 Kad 在安全领域或效率方面进行改进。在区块链中，比特币被设计成使用 DHT 网络，在该网络之上的应用层再进行数据加密来确保系统的安全。也有一些研究是通过重新设计通讯协议来实现安全需求。分布式哈希表 DHT 是非常有效的分布式路由系统，但容易受到恶意节点的影响，此类系统的设计者希望将它们用于开放网络，但对手可以通过，在

网络中引入大量恶意节点来执行女巫攻击，从而大大降低其性能[4] 。也有学者指出，我们还发现了限制这些 DHT 有效性的实施错误、设计问题和安全问题，并为其改进提供了可能的解决方案[5] 。另一个国外的研究方向是提升 DHT 的传输效率。本文也将研究重点关注在效率上，通过 GoLang 语言的协程重写 DHT，以达成提高系统效率的目的。

1.4 设计目标

本文希望基于 Kademlia 模型和 GO 的协程设计理念，将 Kademlia 模型做出个别修改，设计新的网络通信协议，协议中包含规则。

系统内将包括通信协议，路由表，路由算法等基础组件，还要完成加入网络，查询节点，查询资源，下载资源等用户级功能。

在测试中可以验证查找资源功能。同时使用第三方提供的包，将第三方的 Torrent 下载器依赖的 DHT 协议修改成本设计。运行测试效果。

1.5 使用的开发平台

开发过程中需要使用以下软件或开发平台帮助实现本作品。

1.5.1 Linux&Windows

在设计过程中，Windows 系统作为开发环境，Linux 也作为 GO 语言的测试环境。使用 GO 语言时，需要配置多个环境变量。

1.5.2 GoLand

在代码实现环节，需要用到专业的 GoLand 集成开发环境。GoLand 是目前最好用的集成开发环境之一。使用该工具可以省去很多配置工作。软件提供实时保存代码功能省去保存操作。软件提供的代码自动补充功能也省去了很多工作量，也为代码提供了准确性。

第 2 章　DHT 相关技术分析

在引入拓扑结构之前，在公网中的节点如果希望在无服务器的情况下相互访问，每个节点都需要记录网络中所有已知节点的 IP 地址，并且想要访问资源时，需要向每一个已知节点查询资源。因此无服务器网络需要一种机制让网络中的节点需要保存一定数量的节点，在查询资源时也可以得知资源的大致方向，发出带有方向的查询。一般采用的方案是将路由表映射到一个拓扑结构中，根据拓扑结构带来的方向性，让节点也带有方向性，使得节点可以根据方向查询资源，而不是漫无目的的发送广播占用资源。

常见 DHT 拓扑结构有一致散列，二叉树等。使用拓扑结构的分布式网络一般也采用以下设计：使用资源的散列值作为 Key，数据作为 Value。通常采用 SHA-1 算法得到的值作为资源 Key，采用 Node ID 与 InfoHash 同构的设计。由于资源保存在节点上，访问资源与访问节点对访问网络对于物理线路并无差别，也方便路由计算。网络采用某种拓扑结构，比如一致散列，二叉树，多维笛卡尔环面等。

基于不同的拓扑结构设计出的网络，有各自的路由策略。使用最多的 DHT 有 Chord 和 Kademila。前者应用在 AWS 亚马逊云服务的 Dynamo 基础存储架构，后者应用在磁力链接和比特币当中。

2.1 Chord

Chord 诞生于 2001 年，是最早的 DHT 协议的代表设计之一。Chord 采用一致散列作为网络结构，它是一个环形拓扑结构。通过将键与每个数据项相关联，并将键/数据对存储在键映射到的节点上，可以在 Chord 之上轻松实现数据定位，当节点加入和离开系统时，Chord 可以有效地适应，即使系统不断变化，也可以回答查询[6] 。在 Chord 中，散列值按大小连成一个环形结构，散列值的长度决定该网络的路由大小。拓扑结构被规定一个顺时针或逆时针作为路由的移动方向。Node ID 和 InfoHash 采用同构设计，将节点和资源映射到环上。如图 2.1 所示。

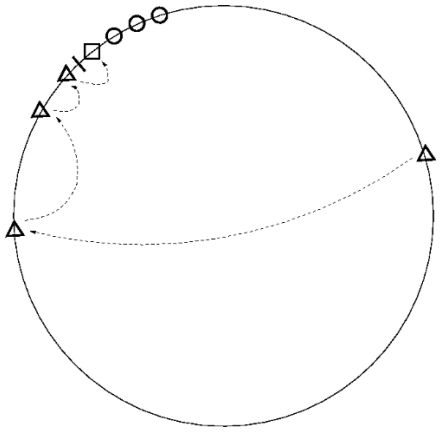


图 2.1 Chord 环形路由

2.2 Kademlia

Kademlia 是在 2002 年被设计出的结构，它使用二叉树作为网络路由的结构。使用异或运算计算节点与节点间，节点与资源间的距离。在二叉树中反应为深度或者跳跃树的层数，网络中的节点之间互相通过节点查找进行信息交换，Kad 节点之间使用 UDP 进行通信，参与的节点形成一个虚拟或覆盖网络。

2.3 探讨不同 DHT 结构优劣

分布式网络中 DHT 网络成为分布式网络共同采用的设计后，Kad 结构的网络再实际应用中也成为主流结构。除了 AWS 亚马逊云服务的 Dynamo 基础存储架构使用 Chord 结构，其余大型著名分布式系统均采用 Kad。无疑在实际项目中，Kad 更被大家认可。本文认为 Chord 在设计层面就劣于 Kad。DHT 网络一般在以下几个维度进行性能比较：

2.3.1 耦合性

Kad 可以根据使用场景来调整桶的个数，一个桶维护一个路由表，路由表的个数是可以灵活改变的，路由表个数通常使用 K 值来表示，对路由表个数的调整不会影响其它代码的实现，耦合度非常低。将整个 DHT 复制并发，两个系统之间也不会发生干扰。

相比之下 Chord 及其它 DHT 的结构作为一个整体一致性太强，机制互相依赖。如果将整个系统并发，两个系统之间也可能发生干扰。

2.3.2 结构实现难度

分布式哈希表 DHT 中的 CAN 网络采用的拓扑结构基于多维笛卡尔环面。笛卡尔环面结构太过于复杂，无论其理论效率如何，代码实现难度极高。由于代码量大，致使运行效率也受影响。

Kad 基于二叉树的拓扑结构，Chord 采用环形队列。后两种结构都很容易被程序实现。同时 Kad 的距离算法最为简单，节点 ID 或 InfoHash 的二进制异或运算计算非常便捷。

2.3.3 并发难度

Kad 受益于代码的低耦合度，路由表可以直接添加成并发进程，同时运行多个路由表查询，彼此之间并不会影响。在查找节点时，系统本身就预设了并发参数为 3，后续也可以在系统自行配置调整。

Chord 及其它 DHT 都没有便捷的角度在代码中实现并发。由于公网上的线路极不稳定，传输速率时快时慢，并发是分布式哈希表所需要的。Kad 路由请求支持并发，不论是 K 值可以改变，还是直接向多节点发出请求，都可以轻易用代码实现。

2.3.4 安全性

假设对 Chord 网络的节点 A 发起攻击，攻击者可以先获得此节点 A 的 ID。攻击者就可以在网络中运行起若干个恶意节点。恶意节点加入 Chord 网络后，会被自动添加到目标 A 的路由表中。如果过多的恶意节点加入到 A 的路由表中，恶意节点可能接到 A 传来的路由请求。攻击者控制这批恶意节点，可以对节点 A 传输含有木马的数据，也可以发起其它类型攻击。

从理论上讲，相同的思路对 Kad 一样有效，毕竟系统本身设计并没有添加身份验证或者 RSA 等加密验证，但是对 Kad 发起攻击的难度非常大。Kad 协议中，节点每次使用功能时，协议要求先 PING 在线节点看是否存活再进行操作。节点被要求每隔一定频率再发出 PING 测试节点是否在线。接收到其它节点传来的指令时，节点被要求将对方加入路由表中。多次筛选规则使留下的节点，在线时间长的节点才容易被其它节点保留。所以攻击者的恶意节点加入目标路由表也需要一定时间。一个节点会保存 8 个路由表，

而一个恶意节点只能加入一个路由表，节点采用到恶意节点所在路由表的几率是 0.125。Kad 协议查询时并发量为 3，每次会向 3 个节点发出查询请求。所以 3 个节点中，只有一个恶意节点，提供的路由结果很容易被其它节点返回结果替代。如果 3 个节点全部都是恶意的，可以截获目标的路由并发起攻击。总体下来攻击者即便在获取到目标 ID 后，需要建立多个节点，也要付出足够的时间成本。这种情况的攻击成本和复杂度将是巨大的，但依然有可行性。

这种攻击被称为 Sybil 攻击。如果没有逻辑上的集中权限管理，除非在资源平等和实体间协调的极端和不切实际的假设下，Sybil 攻击总是可能的[6] 。

第 3 章　系统模型设计

本系统设计的模型基于 Kademlia 经典的模型。我们的系统使用一种新颖的基于 XOR 的度量拓扑结构来路由查询和定位节点，从而简化了算法[8] 。该模型基于传输层的 UDP 协议，在应用层使用自己的通信协议，这个协议除了对传输数据包要求标识协议类型，传输必须的信息，还对系统传输流程做出要求，要求系统也要配合其它节点做出回应。协议中同时规定了节点总是保存更多距离自己近的节点，距离自己远的节点保存较少，这种设计使每个节点不需要保存全部的路由表，只需要维护离自己近的路由表，而且整个系统可以达到完全覆盖。

3.1 拓扑结构——二叉树

在该模型中的网络拓扑结构选用二叉树。将节点与资源用过一定预处理映射在二叉树中。

3.1.1 预处理

网络中的节点 Node ID 和资源 InfoHash 加入网络前需要同构处理。同构处理的节点与资源可以在网络中相同对待，采用相同的距离计算。处理规则是：

（1）Node ID 被设计成随机生成的 160bits。

（2）InfoHash 的预处理使用 SHA-1 算法，将文件经过 SHA-1 算法计算后的值，以二进制形式表示。

3.1.2 散列值映射

节点与资源预处理过后，映射在二叉树中，每一个节点与资源映射后是二叉树的一个子叶。映射规则是：

（1）把节点与资源的散列值以二进制形式表示。

（2）按高位到低位进行的顺序，将二进制的第 n 个 bit，对应着二叉树的第 n 层。

（3）规定划分子树的方向，该层的二进制数是 1 进入右子树，0 进入左子树，此规则也可以做出更改。

（4）直到将散列值全部映射完，资源和节点则对应在二叉树上的一个叶子。

3.2 距离算法

模型计算距离使用 XOR 按位异或运算来计算节点与节点，节点与资源间的距离。这种 XOR 按位异或运算存在以下特性：

（1）XOR 符合交换律，具备对称性。参照公式 3.1。

$$(A \oplus \mathrm{B}) == (B \oplus \mathrm{A}) \tag{3.1}$$

（2）反身性，自身距离为零。参照公式 3.2。

$$(A \oplus \mathrm{A}) == 0 \tag{3.2}$$

（3）三角不等式。参照公式 3.3。

$$(A \oplus \mathrm{B}) + (B \oplus \mathrm{C}) \geq (A \oplus \mathrm{C}) \tag{3.3}$$

这三种特性使得模型更接近于物理路由线路。在物理线路中，模型内自己到自己的距离为 0，自己到对方的距离和对方到自己距离相同，通过两次查询访问节点比自己访问节点距离远。

3.3 路由机制

在该模型的网络中，系统希望保存更多距离自己近的节点，保存少量距离远的节点，这个过程在二叉树中通过二叉树拆分，将节点与自己的距离进行分组。越早拆分的子树距离自己越远，包含的节点越多。得到拆分后的子树中，每一个子树下的节点之间距离较近，可以近似将它们视为一个网络，因为它们之间相互保存的路由信息更多。节点只需要记录较远距离路由网络中的一个或几个节点，通过那个节点就可以查询访问它所在的子网络。

3.4 二叉树拆分

对每一个节点，节点按照自身的视角对整个二叉树进行拆分。如图 3.1。拆分按照以下规则递归：

（1）从根节点开始，把异或值等于 1 的子树拆分。

（2）在剩下的子树中再次进行（1）操作，递归直到剩余节点自身停止递归。

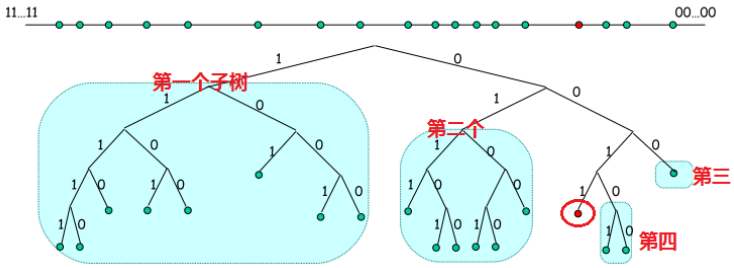


图 3.1 二叉树拆分

系统缺省使用 160bits 作为散列值，拆分将得到最多 160 个子树。一个节点拆分子树后，每一个子树下的节点以它们自身视角，它们之间距离较近，因此将它们视为一个路由网络。节点只需要记录较远距离路由网络中的一个节点，通过那个节点查询访问整个路由。

3.5 路由表

在 Kademlia 模型中，每个节点只需要储存子树里的每一个节点，组成起来的网络相互访问，就可以实现访问全部节点。由于异或运算带来的路由跳数与物理距离跳数相近，在物理层面拆分的路由也是相近的。但是考虑到网络链路的复杂与不稳定性，系统需要健壮性。通常路由需要保存一定冗余。Kademlia 设定了 K 桶（K-bucket）机制。每个节点在完成子树拆分后，要记录每个子树里面的 K 个节点。K 桶代表着路由表。每个子树映射一个路由表，拆分出 n 个子树则对应了 n 个路由表，每个路由表最多含有 K 个节点。K 是一个变量，通常设为 8。

但是在此设计中，决定放弃 K 桶设计。使用 K 桶会得到多组路由表，路由表管理时，加载和固化增加代码难度，并且本文认为意义不大。取消 K 桶设计使二叉树直接映射到路由表中。直接在路由表中保存多节点即可。同时 K 桶决定着后续并发任务的选择，取消 K 桶将并发任务更加细分也符合 GO 语言设计思路。

3.6 路由表刷新机制

本模型中，路由表刷新将遵循如下几种方式刷新节点：

1、探测失效节点

系统中规定，系统需要定期 PING 路由表中的每一个节点，删除路由表中离线的节点，并使用 FIND_NODE 补充路由表。

2、主动收集节点

节点主动发起查询节点 FIND_NODE 请求，请求流程中根据路由规则刷新路由表。

3、被动收集节点

节点收到其它节点发来的请求 FIND_NODE 或 FIND_PEERS，将对方加入自己的路由表中。

3.7 并发请求

如果使用 Kademlia 的 K 桶设计思路，并发会在 K 桶层面进行设计。同一个 K 桶中的节点都可以代表这个子网的路由，对同一个 K 桶中的节点同时发起查询请求，节点之间不会发生干扰。

在本模型中，因为废弃 K 桶的设计思路，需要将并发写在具体的在路由表中筛选节点的任务中。筛选过程中，直接在路由表中计算获得 K 个需要的节点。一同添加为 GO 协程。

3.8 协议消息

根据开源协议 Bittorrent 中 BEP 5 规范，协议将使用 PING，FIND_NODE，GET_PEERS，ANNOUNCE_PEER 四种指令。

（1）PING 用于检查节点是否在线。

（2）FIND_NODE 查找节点。收到 FIND_NODE 后若无目标节点，则返回 K 个最近的节点。

（3）GET_PEERS 查找资源。收到 GET_PEERS 后若无目标资源，则返回 K 个最近的节点。

（4）ANNOUNCE_PEER 通知其它节点存储资源键值对。

3.9 流程控制

模型还需要流程规则控制，通过流程控制，使得各个节点在传输过程中交换自己的部分路由信息，使整个网络中的路由表都在不断更新。

3.9.1 节点查询

节点查询时，需要遵守以下规定：

（1）首先由发起者确定目标 ID 对应二叉树中的位置，然后从自己的路由表中筛选出 K 个距离目标 ID 最近的节点，并同时向这些节点发起 FIND_NODE 的查询请求。

（2）被查询节点收到 FIND_NODE 请求后，从路由表中挑选距离目标 ID 最近的 K 个节点，回复给发起者。

（3）发起查询的节点在收到返回的 K 个节点后，将这些节点更新到自己的路由表中。再次从路由表中挑选距离目标节点 ID 最近的 K 个节点，并做二次筛查，仅挑选未发送请求的节点重复步骤（1）。

（4）重复上述步骤直到搜索到节点。若返回的 K 个节点没有比路由表中节点距离目标节点更近的节点，路由表不再更新。这时就表示未找到目标节点，查询停止。

在查询过程中，没有回复 PING 的节点，会被从路由表中删除。在整个过程中，发起查询的节点必须在查询前先使用 PING 来保证最终获得的 K 个节点都是在线的。

3.9.2 节点定位

当节点要查询资源键值对时，需要遵守以下规定：

（1）发起节点会查找自己路由表是否存储了资源的键值对数据，如果存在则直接返回，否则就返回 K 个距离资源键最近的节点，并向这 K 个节点 ID 发起 GET_PPERS 请求。

（2）收到 GET_PEERS 请求的节点，首先也是检查自己是否存储了资源的键值对，如果有直接返回值，如果没有，则在自己的路由表中返回 K 个距离资源键最近的节点。

（3）发起者如果收到资源值，则结束查询过程。否则发起者在收到这些节点后，先将收到的节点存入路由表，并再次从路由表中挑选距离目标节点最近的 K 个节点，对节点进行判断，判断出未发送请求的节点，对它们发起 GET_PEERS 请求。

（4）不断重复上述步骤，直到获取到资源值。或者无法获取比路由表中节点距离资源键更近的活动节点，这时就表示未找到资源值。

在后续的流程控制中规定，如果上述查找最终找到资源，则在没有返回资源值的最近节点上也会缓存资源键值对，这样下次再查询相同的资源键时就可以加快查询速度。

3.9.3 保存资源

当节点收到一个键值对的数据时，需要遵守以下规定：

（1）发起者首先定位 K 个距离资源键值最近的节点。

（2）然后发起者对这 K 个节点发起 ANNOUNCE_PEER 请求。

（3）接收到 ANNOUNCE_PEER 请求的节点将保存键值对数据。

（4）执行 ANNOUNCE_PEER 操作的 K 个节点需要发出 ANNOUNCE_PEER 每小时一次，值为自己所有的资源键值对。

（5）资源键值对在一个节点发出 ANNOUNCE_PEER 后 24 小时过期，用于淘汰掉失效信息。

3.9.4 加入网络

一个新节点加入网络时，需要遵守以下规定：

（1）新节点 A 首先需要一个种子节点 B 作为引导，并把该种子节点加入到对应的路由表中。

（2）首先生成一个随机的散列值作为节点 ID，直到离开网络，该节点会一直使用该 ID。

（3）向节点 B 发起 FIND_NODE 请求，FIND_NODE 的值为节点 A 的 Node ID。

（4）节点 B 在收到节点 A 的 FIND_NODE 请求后，会根据 FIND_NODE 请求的约定，找到 K 个距离 A 最近的节点，并返回给 A 节点。

（5）A 收到这些节点以后，把这些节点加入到自己的路由表中。

（6）然后节点 A 会继续向这些刚拿到的节点发起 FIND_NODE 请求，如此往复，直到 A 建立了足够详细的路由表。

3.9.5 退出网络

模型中的节点退出 DHT 网络不需要任何操作，其它 DHT 节点通过定时刷新来刷新掉节点。

第 4 章　系统代码开发

本系统的实现需要基于UDP之上，设计整体网络拓扑结构，远程过程调用协议RPC，路由表，路由表生成，节点加入退出机制，资源搜索保存机制。

通过 GO 语言实现模型。代码工作主要在实现 DHT 的树结构等数据结构，远程过程调用协议 RPC，路由表，路由表生成和多线程通信等功能。在主函数中，需要完成加入网络，搜索节点，搜索资源，下载资源，退出网络五个功能。主函数中，DHT 对象将被设置为并发，通过向节点之间发送管道信号量来完成通讯和流程控制。每个本机开启的并发节点被视为是独立的调度对象。节点之间存在一定共享数据，节点也存在自己开辟的内存不可被其它节点访问。节点与节点，节点与主程序之间的通信被设置成非阻塞式。

4.1 实现原则

本系统原型设计遵循大部分 Kad 协议。通信协议完全遵守 BEP 5。这么做带来的好处是可以最大限度与现存网络兼容，但弊端是可改进的设计点不多。本文针对在内存中处理的二叉树和并发方面做出改进设计。二叉树中选择废弃掉 K 桶数据结构。在 Kad 原型中，二叉树会被映射到 K 桶对象中，K 桶再映射到路由表中。而本文选择废弃掉 K 桶对象，直接从二叉树中搜索数据映射到路由表中。这么做虽然会使系统整体耦合度增加，但是使网络的结果变得清晰。在效率方面，这么做并不会使运行效率有直观的体现，至少是在本系统中对效率的影响可以忽略不计，同时这样处理更利于做二次开发。在并发方面，由于 K 桶对象的取消，并发并不在对 K 桶对象实现。本系统由于使用 GO 提供的协程并发，理论上应把操作细分，因此对二叉树并发更符合设计理念。站在软件工程的角度，对路由表添加并发更符合原设计的理念，因为那样做在软件工程上耦合度低。但是在本设计中，使用 GO 语言设计的目标是提升效率，因此打算在工程上做出牺牲，选择增加设计难度和耦合度，来提升运行效率。

4.2 系统开发环境配置

本系统使用 GoLang 编程语言，又称 GO 语言。具有静态强类型，编译型的特点。语言在并发特性上，采用将线程细分为协程的方式，使得协程轻量且高效。语言提供了 LRQ 本地运行队列协助操作系统自动管理协程。因为该语言在网络通信方面性能突出，因此也被本文所采用。

考虑运行效率，GO 没有使用像 Java 语言一样的通用虚拟机配置，而是需要手动搭建配置。使用时需要在系统环境变量中写入开启 CPU 内核数，是否开启包管理 Gomod 模块，以及代理服务器。

整体项目因为最终作为包提供给开发者后续编程使用，同时整体项目规模庞大，项目本身也需要引用编码工具，网络工具等第三方开发包，因此需要使用语言自带的 Gomod 功能来对项目整体进行管理。Gomod 模块使用类似于 Java 中的 Maven 项目，需要在.mod 配置文件中写入引用包的包名称，包发布地址，版本号信息。在执行编译命令时，脚本将会在网站上自动下载引用包。

为了开发便捷，本项目使用了 GoLand 集成开发环境来帮助项目完成，使用该集成开发工具可以更便捷地进行模块测试。

4.3 数据结构 Data Structure

系统需要建立的数据结构有 DHT，Config 等。DHT 是系统的路由表。在 GO 语言中，结构体可同时作为对象使用。该对象还需要作为一个并发的协程。由于只使用少数数据结构，并且存在指针关系，因此不能设计成关联数据库。Config 是存储系统配置相关信息的对象，里面主要记录系统在本机的 IP 地址端口号，系统调整路由表并发量，接收请求并发量，接收数据并发量以及 PING 节点的心跳频率等信息。

4.3.1 Type DHT

本系统中使用相同长度对 Node ID 和 InfoHash 处理。对于资源处理的 InfoHash 获取，流行的哈希散列的算法有 MD 和 SHA 两种。其中 MD 算法更多用于文件验证，在性能上 MD 计算效率快于 SHA，在安全上 SHA 抗攻击能力更强。在本系统中哈希值将被使用于资源定位，哈希值要确保资源唯一，并在网络中传输，所以在安全方面需求更

高，本文采用 SHA-1 算法。对于 Node ID，只需要设计将 IP 地址和端口转码即可。系统中使用 Bencode 编码，直接处理端口号。Bencode 编码不在本文研究范围内，不做代码重写，系统将使用他人开源的包。哈希长度的取值上，取值太小会使节点或资源地址发生碰撞，会引发安全问题；值太大又会使路由表过大，但节点数量在总系统中含量太低，使树种节点空散，达不到组网的目的。本系统中将采用 160bits 作为散列值长度。

表 4.1 type DHT struct

字段名	数据结构
nodeID	string
config	Config
routingTable	*routingTable
peerStore	*peerStore
exploredNeighborhood	Bool
remoteNodeAcquaintance	chan string
peersRequest	chan ihReq
nodesRequest	chan ihReq
pingRequest	chan *remoteNode
portRequest	chan int
removeInfoHash	chan InfoHash
stop	chan bool
Wg	sync.WaitGroup
clientThrottle	*nettools.ClientThrottle
store	*dhtStore
tokenSecrets	[]string
PeersRequestResults	chan map[infoHash][]string
Logger	Logger
DebugLogger	DebugLogger

DHT 数据结构用于建立核心的 Kad 树型结构。参照表 4.1。DHT 创建函数与 Config 创建函数合并为 New 方法。本设计在此对象中提供了一些工具方法以便后续操作。核

心功能为 AddNode 函数和 GetPeers 函数，添加节点函数实现了将新节点添加到路由表中。获取节点函数则是通过发送 GET_PEERS 协议来向路由表中最近的节点发送查询请求。在节点挂载的协程中处理返回结果，不在此函数中处理。

在这个结构体中，提供了 DebugLogger 接口和 Logger 接口，以便其它开发者拓展其功能。DebugLogger 接口提供打印日志文档接口。Logger 和接口允许 DHT 客户端为某些 RPC 提供钩子函数。

4.3.2 Type Config

数据结构 Config 主程序运行的配置文件。Config 存储系统在本机的 IP 地址端口号，系统调整路由表并发量，接收请求并发量，接收数据并发量以及 PING 节点的心跳频率等信息。Config 可供用户修改，也可以由系统自动生成。参照表 4.2。

表 4.2 type Config struct

字段名	数据结构	默认值	说明
Address	string	自动获取	—
Port	int	自动获取	—
NumTargetPeers	int	5	DHT 将尝试为每个正在搜索的信息哈希找到的节点数量
DHTRouters	string	—	启动网络 DHT 的路由器
MaxNodes	int	100	在路由表中要存储的最大节点数
CleanupPeriod	time.Duration	15	心跳时间
SaveRoutingTable	bool	true	每隔几分钟在磁盘上保存路由表快照
SavePeriod	time.Duration	5min	保存路由表到磁盘的频率
RateLimit	int64	100	每秒要处理的最大数据包
MaxInfoHashes	int	2048	infoHash 的数量
MaxInfoHashPeers	int	256	每个 infoHash 要跟踪的节点的数量
ClientPerMinuteLimit	int	50	忽略请求最大值的数据包/min
ThrottlerTrackedClients	int64	1000	记忆的主机数量
UDPProto	string	—	—

Config 对象中提供了初始化对象的核心功能。主要为各种数据结构开辟内存地址，调用日志接口，并调用存储模块的 SaveStore 方法。目前 Config 并没有固化在磁盘上，系统提供 New 函数来生成默认 Config 文件。后续希望将 Config 写成私有对象，完全由系统生成，不提供接口给用户修改。

4.4 散列值映射实现

本系统采用算法把 InfoHash 映射到一个二叉树，映射过程将 InfoHash 以二进制形式表示。从高位到低位，使用 For 循环遍历各个位，二进制的第 n 个 bit，对应二叉树的第 n 层。第 n 层的值为 1 则映射在值为 1 的右树，直到将散列值遍历完称。InfoHash 就映射在二叉树上的一个叶子上，完成映射。

4.5 路由表实现

DHT 路由使用二进制树。节点的 ID 为 20 字节。所有的节点都插入二叉树中，高度固定为 160（20 字节）。为了查询一个 InfoHash，使用每一级的 InfoHash 位做一个无序遍历。一个节点和一个 InfoHash 之间的距离是各自字符串的 XOR。

4.6 远程过程调用协议 RPC 实现

相关开源组织给出的协议规范中，规定了网络中使用 PING，FIND_NODE，GET_PEERS，ANNOUNCE_PEER 四种协议类型。为了将系统与其它语言以及开源资源相互兼容，本系统采用与协议规范中相同的协议类型，传输使用 JSON 数据包可以兼容。

4.6.1 PING 实现

查看节点是否可达，并将其保存在路由表中。调用 PING 函数会判断配置信息是否为空，调用 PingNode 函数。

在 PingNode 函数中会判断传参是否为空，指针是否为空，然后调用路由表中的函数判断路由距离。然后开辟内存空间，写入‘ty’协议名称为‘ping’，‘transID’，‘nodeID’，‘infoHash’等字段在 JSON 数据包中，调用系统接口函数发送数据到目标并调用日志接口。

4.6.2 FIND_NODE 实现

当 DHT 节点数下降时运行 FIND_NODE，或每隔一定时间运行一次，为了刷新 DHT 路由表的节点，FIND_NODE 函数会首先判断 Config 中相关信息是否为空，如果不为空，调用路由表函数获取已知节点并对每一个节点执行 FindNodeFrom 具体实现函数。

程序进入 FindNodeFrom 函数，首先判断传参是否为空，指针是否为空，然后调用路由表中的函数判断路由距离，然后开辟内存空间，写入‘ty’协议名称为‘find_node’，拼接‘transID’，‘nodeID’，‘infoHash’等字段在 JSON 数据包中，调用系统接口函数发送数据到目标并调用日志接口。

```
func (d *DHT) findNodeFrom(r *remoteNode, id string) {
    ty := "find_node"
    transId := r.newQuery(ty)
    ih := InfoHash(id)
    d.DebugLogger.Debugf("findNodeFrom adding pendingQueries transId=%v ih=%x", transId, ih)
    if _, ok := r.pendingQueries[transId]; ok {
        r.pendingQueries[transId].ih = ih
    } else {
        r.pendingQueries[transId] = &queryType{ih: ih}
    }
    queryArguments := map[string]interface{}{
        "id":        d.nodeId,
        "target": id,
    }
    query := queryMessage{transId, "q", ty, queryArguments}
    d.DebugLogger.Debugf("DHT sending find_node. nodeID: %x@%v, target ID: %x , distance: %x", r.id,
r.address, id, hashDistance(InfoHash(r.id), ih))
    r.lastSearchTime = time.Now()
    sendMsg(d.conn, r.address, query, d.DebugLogger)
}
```

4.6.3 GET_PEERS 实现

为资源寻找节点。迭代查询 DHT 节点并找到新的来源，为一个特定的信息哈希找到新的来源。实现同 FIND_NODE，传参 InfoHash。函数首先将 InfoHash 解析，然后判断端口 IP 以及 Config 等是否为空，如果不为空则执行 GetPeersFrom 具体执行函数。

在 GetPeersFrom 函数中，首先判断传参是否为空，指针是否为空，然后开辟内存空间，写入 ty 协议名称为‘get_peers’，判断哈希值安全后，拼接 InfoHash 等字段在 JSON 数据包中，调用系统接口函数发送数据到目标并调用日志接口。

```
func (d *DHT) getPeersFrom(r *remoteNode, ih InfoHash) {
    totalSentGetPeers.Add(1)
    ty := "get_peers"
    transId := r.newQuery(ty)
    if _, ok := r.pendingQueries[transId]; ok {
        r.pendingQueries[transId].ih = ih
    } else {
        r.pendingQueries[transId] = &queryType{ih: ih}
    }
    queryArguments := map[string]interface{}{
        "id":          d.nodeId,
        "info_hash": ih,
    }
    query := queryMessage{transId, "q", ty, queryArguments}
    sendMsg(d.conn, r.address, query, d.DebugLogger)
}
```

4.6.4 ANNOUNCE_PEER 实现

该方法用于与此节点保存下载资源时使用，节点会将资源保存并发送给网络中距离最近的 8 个节点上备份。

4.7 用户级功能

系统设计了用户层为使用者提供便捷的使用方法。在协议中规定，节点在加入网络，查找资源，查找节点，下载资源时，都需要做出一系列流程配合网络其它节点，用来相互更新路由表。这些流程都是通过组装远程过程调用协议 RPC 来完成的。

4.7.1 加入网络实现

系统启动后并没有加入 Kad 网络，因此需要执行一个启动流程以完成初始化，启动时需要执行以下流程：

（1）自身节点加入 Kad 网络时，必须要得知一个已知节点作为引导，系统中借助网上用于帮助 Kad 网络的服务器来获取一个节点。

（2）系统自身作为一个节点生成一个随机的散列值作为 Node ID，直到程序退出时会删除 ID。

（3）系统向已知节点发送 FIND_NODE 请求查找节点，请求的值是自身。

（4）已知节点收到 FIND_NODE 请求后会执行 FIND_NODE 请求的返还方法，该方法返回距离本系统最近的 8 个节点。

（5）系统判断路由表是否达到一千个节点，如果满则退出加入网络方法，如果不满则将收到节点后加入二叉树和路由表，并继续对已知节点执行步骤（3）。

4.7.2 退出网络实现

原先 Kad 模型中的节点退出时不需要任何操作，在本系统中沿用此设计。由网络中其它的节点通过定时刷新等机制来刷新掉离线节点。

4.7.3 查找节点实现

系统执行查询节点功能时，并发数量一般设为 3，节点将执行以下流程：

（1）发起请求的节点根据目标的 Node ID 在二叉树中找到对应位置，然后在二叉树中找它所属于的子树，然后根据异或运算在路由表中计算出 8 个距离最近的节点并发起 PING。

（2）节点根据 PING 返回结果进行判断，如果有节点未响应，则证明该节点已离线，在路由表中删除该节点，再向路由表中索要 8 个节点，对这些节点发起 PING 请求。如果节点全部响应，则对这些节点发起 FIND_NODE 请求。

（3）其它节点收到 FIND_NODE 请求后，将执行步骤（1）（2）。

（4）节点发起请求后需要启动一个阻塞，阻塞进行判断，如果收到资源，更新自己的结果列表。并再次从其中距离目标节点最近的 8 个节点，依次判断是否发送过请求。挑选未发送请求的节点，依次判断是否发送过请求，如果节点未发送过则发送 FIND_NODE 请求。如果所有节点都已经发送过请求，则证明网络无法再获取到距离更近的节点了。此时该方法将关闭。

4.7.4 查找资源实现

当系统执行查询资源时，节点将按照下述流程执行：

（1）发起请求的节点首先进行判断，查找自己是否存储了 InfoHash，如果存在则直接返回，否则返回 8 个距离 InfoHash 最近的节点，并向这 8 个节点发起 FIND_PEERS 请求。

（2）其它节点收到 FIND_PEERS 后，执行步骤（1）。

（3）节点发起请求后需要挂起一个阻塞，阻塞进行判断，如果收到资源，则节点关闭查询过程。否则接收到的为 8 个节点，接着将这些收到的节点更新自己的路由表。再对其中 8 个距离最近的节点，依次判断是否发送过请求，如果节点未发送过则发送 FIND_PEERS 请求。如果所有节点都已经发送过请求，则证明网络无法再获取到距离资源更近的节点了。此时该方法将关闭。

4.7.5 保存资源实现

当系统需要保存资源时，是已经得到了查询资源返回的值了。系统保存时还需要执行以下流程来保证整个网络可以有效运行：

（1）节点保存时将先对距离资源最近的 8 个节点发起 PING 请求。

（2）如果未得到回应，则在路由表中删除该节点。如果得到节点返回的 PING，则对这 8 个节点发起 ANNOUNCE_PEER 请求。

（3）节点接收到 ANNOUNCE_PEER 请求后，将资源保存。

4.8 系统主函数

系统的入口提供了 DHT Start 函数启动 DHT 节点，该函数含有判断是否有 Config 并调用初始化 Config 的功能。

调用 DHT Start 函数会在本机地址上启动一个监听器，抓取错误，添加协程组，一个单独的 GO 协程中运行 Loop。如果 DHT 成功启动则返回 Nil，否则返回错误。

```
func (d *DHT) Start() (err error) {
    if err = d.initSocket(); err == nil {
        d.wg.Add(1)
        go func() {
```

```
            defer d.wg.Done()
            d.loop()
        }()
    }
    return err
}
```

其调用的 Loop 函数也是主要函数。Loop 负责启动路由表，监听返回的 Requests 数据包，直到 Loop 被另一个协程使用 Stop 关闭。

在一个 Loop 中，首先关闭端口，然后开辟内存空间存储各种数据结构，重新开启端口，获取系统时间，接收数据包，调出 Config 配置，装载最大数据包限制配置，判断是否超出最大数据包量。将 DHT 装载在 IP 和端口上，进入无限循环，进入 Swich 循环，判断如果收到 Stop 信号，退出。如果收到 FIND_PEERS，判断是否存在 InfoHash，存在则返回 InfoHash；如果不存在，则返回距离资源最近的 8 个节点。如果收到 RemoveLocalDownload 信号，则删除下载内容。如果收到 PING，返回执行 PING。在收到主函数的 ProcessPacket 时，进入 ProcessPacket 函数并执行。

其中调用的 ProcessPacket 函数为主函数发送的信号量。其中做两次 Switch 接收信号量的判断，一次为主动调用，一次为被动接收。

```
func (d *DHT) processPacket(p packetType) {
switch r.Q {
        case "ping":
            d.replyPing(p.raddr, r)
        case "get_peers":
            d.replyGetPeers(p.raddr, r)
        case "find_node":
            d.replyFindNode(p.raddr, r)
        case "announce_peer":
            d.replyAnnouncePeer(p.raddr, node, r)
        default:
            d.DebugLogger.Debugf("DHT: non-implemented handler for type %v", r.Q)
        }
    default:
        d.DebugLogger.Debugf("DHT: Bogus DHT query from %v.", p.raddr)
    }
}
```

系统也提供了 Stop 函数关闭系统。

4.9 协程 Goruntine

节点的操作被添加到 GO 的协程中，协程是更细分的线程。协程并没有增加线程数量，只是在线程的基础之上通过分时复用的方式运行多个协程，而且协程的切换在用户态完成，切换的代价比线程从用户态到内核态的代价小很多。语言本身对计算机处理器设计了 LRQ 本地运行队列来管理协程。本地运行队列会对计算机处理器的进程进行调度，使处理器内核之间负载均衡，以提升效率。语言中只需要对方法进行字段标记就可以添加入协程，定义信号量，由系统自动管理回收。

```
go func()
```

本系统添加并发的角度非常直观。同一个路由表中的每个节点都是平等的。对同一个路由表中的节点发起请求，互相之间没有影响，几乎无耦合[9] 。

4.10 存储

存储使用本系统实现的 DhtStore 模块将路由表固化在磁盘上。该对象记录了 Node ID，InfoHash 和磁盘位置。对象内提供了调用系统存储接口，选择磁盘路径，保存文件的方法实现。

第 5 章　测　试

为了验证 DHT 系统是否能被运行，同时测试与公共 DHT 的兼容性，本文编写了测试函数 Test 和下载函数 Download。测试函数主要用于测试查找资源功能。下载函数主要用测试保存资源功能。

5.1 测试查询资源

在测试函数中，主要测试查找资源功能，收集节点信息，在路径中保存文件地址，控制台中打印 10 条返回的节点。搜索网上的任意的 InfoHash。

infoHash= 0001022025414243205859 5a6162637f8081feff

在控制台中启动测试函数，测试结果得到了 10 个节点。在控制台中打印出结果。如图 5.1。在 Wireshark 中抓包获得了通讯的数据包。如图 5.2 所示。

```
PS F:\GOPATH\bin> .\test.exe  0001022025414243205859 5a6162637f8081feff
============================ DHT
Peers found:
0: 195.154.172.169:54949
1: 195.154.172.169:54949
2: 195.154.172.169:54949
3: 195.154.172.169:54949
4: 195.154.172.169:54949
5: 195.154.172.169:54949
6: 195.154.172.169:54949
7: 195.154.172.169:54949
```

图 5.1 终端返回结果

ip.dst== 195.154.172.169

.	Time	Source	Destination	Protc
263	-33.167151	172.16.107.184	195.154.172.169	UDP
1873	-28.245212	172.16.107.184	195.154.172.169	UDP
5577	-20.175699	172.16.107.184	195.154.172.169	UDP
5660	-20.161446	172.16.107.184	195.154.172.169	UDP
6519	-19.226263	172.16.107.184	195.154.172.169	UDP
6874	-17.979198	172.16.107.184	195.154.172.169	UDP

图 5.2 Wireshark 抓包结果

测试结果显示，系统可以按照预设的频率向目标节点发送 PING，FIND_PEERS 数据包，并且有发现节点可以返回数据包。

5.2 测试下载资源

测试下载函数。如果要下载文件，除了需要 DHT 路由用来路由，还需要 SFTP 来建立 TCP 传输文件，传输文件时还会遇到 NAT 问题，导致文件无法建立连接，因此还需要 UPNP 来实现 NAT 穿透。除此之外，下载种子文件还需要兼容种子协议规范的 Tracker 功能，同样需要引入第三方 Tracker 包。本次测试使用的是第三方提供的下载工具，将代码修改，修改 DHT 使用为本系统。在 Ubuntu 官网下载得到官方的 Torrent 种子文件。测试前先使用测试函数测试节点是否在终端内启动函数，得到返回结果。如图 5.3 所示。

```
========================== DHT
Peers found:
0: 202.98.213.218:1
1: 58.134.152.247:1
2: 222.67.14.222:1
3: 42.80.41.245:6881
4: 58.37.36.192:25536
5: 58.134.152.247:55055
6: 61.150.60.223:35775
7: 61.150.60.223:40813
8: 61.178.98.69:10105
9: 82.65.240.27:12958
```

图 5.3 ubuntu 测试查询

在控制台中调用下载函数，得到返回结果。如图 5.4 所示。

```
Have 609 of 12891 pieces 4.724226 % complete
Have 610 of 12891 pieces 4.7319837 % complete
Have 611 of 12891 pieces 4.739741 % complete
Have 612 of 12891 pieces 4.747498 % complete
Have 613 of 12891 pieces 4.7552557 % complete
Have 614 of 12891 pieces 4.763013 % complete
Have 615 of 12891 pieces 4.77077 % complete
Have 616 of 12891 pieces 4.7785277 % complete
Have 617 of 12891 pieces 4.786285 % complete
Have 618 of 12891 pieces 4.7940426 % complete
Have 619 of 12891 pieces 4.8018 % complete
Peers: 58 downloaded: 164003840 (2.02 MB/s) uploaded: 0 ratio:
```

图 5.4 测试下载

在磁盘路径中可以看到下载的文件。如图 5.5 所示。

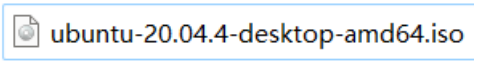


图 5.5 查看文件

结　语

本系统使用 GoLang 语言实现了 Kad 分布式网络系统。在实现过程中，因为协程与主进程的控制使用信号量来控制整体程序实现，是本设计中的最大困难。系统尝试兼容公共 DHT 网络，首先测试了 GET_PEERS 功能，可以获取到节点。通过使用第三方的 SFTP 和 Tracker 包测试下载资源功能可以实现。测试结果运行良好，但是受限于实验材料，DHT 网络需要多台主机和启动服务才能将整体系统运行起来，无法组建自己的网络。本系统受限于个人能力，技术还不能做到 Torrent 或 Magnet 的 DHT 节点完全兼容。如果要搭建自己的分布式网络，则需要启动上百台计算机同时运行，也需要一台启动服务器作为初始路由器将整个网络连接，当节点的路由表节点充足且有活跃节点存在后，系统将不需要服务器，达到完全分布式的状态。

同时在本文第一章中指出，Kademlia 的结构在安全性方面也存在问题，本文任务 DHT 需要设计更优秀的结构，而不是在结构外通过拓展的方式改进。

参考文献

[1] Ma W. Research on how to build new Foreign Trade competitive advantages of Ningbo by using the" Internet+" under the innovation-driven Background[C].//2016 2nd International Conference on Humanities and Social Science Research (ICHSSR 2016). Atlantis Press, 2016: 381-384.

[2] 张玉洁,何明,孟祥武.基于用户需求的内容分发点对点网络系统研究[J].软件学报,2014(1):98-117.

[3] 温安宇. 基于 DHT 的 key-value 分布式存储系统[D].哈尔滨工业大学,2010.

[4] Danezis, G., Lesniewski-Laas, C., Kaashoek, M.F., Anderson, [D]. (2005). Sybil-Resistant DHT Routing. In: di Vimercati, S.d.C., Syverson, P., Gollmann, D. (eds) Computer Security – ESORICS 2005. ESORICS 2005. Lecture Notes in Computer Science, vol 3679. Springer, Berlin, Heidelberg.

[5] Crosby S A , Dan S W . An Analysis of BitTorrent's Two Kademlia-Based DHTs.[J].2007.

[6] Stoica I, Morris R, Liben-Nowell D, et al. Chord: a scalable peer-to-peer lookup protocol for internet applications[J]. IEEE/ACM Transactions on networking, 2003, 11(1): 17-32.

[7] Douceur J R. The sybil attack[C].//International workshop on peer-to-peer systems. Springer, Berlin, Heidelberg, 2002: 251-260.

[8] Maymounkov P, Mazieres D. Kademlia: A peer-to-peer information system based on the xor metric[C].//International Workshop on Peer-to-Peer Systems. Springer, Berlin, Heidelberg, 2002: 53-65.

[9] 黄煜东. 基于 rust 编程语言的 i2p 匿名通信设计[D].华中师范大学,2018.

致　谢

首先我要感谢我的指导老师谢苑老师，在我做毕业设计的过程当中，她负责任的指导和建议使我受益匪浅，她每时每刻的督促使我不敢有丝毫的懈怠，她一直给予我悉心的教导和莫大的支持，无论在学习上还是在精神上都给予了帮助，这些帮助和鼓舞对我而言是一笔财富，她一直鞭策着我认真的完成毕业设计，而且也让我在面对未来的人生舞台时同样充满了信心。在这几个月的时间里，老师对我所犯的错误也给予了恰当的指导和包容，让我能够及时认识到自己的错误并力争改过。在此，我向谢老师致以深深的谢意！

接着还要特别感谢王光辉老师、郭念老师和赵龑骧老师。在我毕业设计的专业知识领域，存在很多我不明白的知识，是这几位老师给了我专业指导。这几位老师并不负责我的毕业设计，同时他们也带领有自己的学生，虽然他们工作繁忙，但依然抽出时间帮助我批阅毕业设计。向他们表达由衷的感谢！

其次还要感谢我的同学们，在我的毕业设计中，他们的指导给予了我极大的帮助，使我对整个毕业设计的思路有了总体的把握，并耐心的帮助我解决了许多实际问题，使我有了很大收获。他们在整个开发过程中提出了许多建设性意见，并给我解决了一些专业性问题，使我受益匪浅。

最后我要特别感谢几年来河南大学软件学院的老师们，他们传授我知识，使我有丰富的知识储备完成毕业设计。更要感谢我的家人们对我学业上的支持和鼓励，感谢所有关心帮助过我的人。

TONYYO 通远股份　　格子达 教育大数据专家

G · 格子达论文检测报告【简版】

报告编号:B5B52B2EA8644FC992301BF6706A3210

送检文档:基于GO语言的Kademlia网络协议的设计与开发

作 者:李嘉恒　　送检单位:河南大学　　送检时间:2022-05-10 20:44:51

比对索引库

1989-01-01至2022-05-10

学术期刊库	报纸资源库	本科论文共享库	格子达公识库
学位论文库	互联网资源库	专利库	机构自建库
会议论文库	格子达多元库		

检测结果

总相似比:5.08%(总相似比=复写率+引用率)



学术检测指标:自写率94.92%复写率1.17%引用率3.91%（含自引率0.0%）

格式检测指标:主体缺失 不规范引用句子数9

其他类型检测结果:去除引用后总相似比:1.17% 同校同届总相似比:0.61%

相似片段:复写片段4 同届片段6 引用片段10

指标名称	学校要求	指标检测结果	系统判定
总相似比	不超过20%	5.08%	符合
论文总字数	不少于1000字/单次	22258字	符合

河南大学论文检测报告 No.3189088

论文信息

论文题目：基于GO语言的Kademlia网络协议的设计与开发　作者：李嘉恒　指导教师：谢苑

全文页数：46 页　字数统计：21171　中文字符：19223　英文字符：1948

检测说明

1.检测依据：学校模板：《河南大学本科毕业论文（设计）模板》，国家标准：《GB7713学位论文编写格式》，《GB7714参考文献著录规则》，《GB15834标点符号用法》，《GB15835出版物上数字用法》，《GB3100国际单位制及其应用》，《GB3101有关量单位符号的一般原则》，《GB3102空间和时间的量和单位》。
2.万字差错率计算方法：错误数*10000/总字数（每万字差错率），参考《图书质量管理规定》。
3.判断依据：万字差错率超过3.00不合格
4.如果您发现检测出现误报或漏报，请在订单咨询处提交问题，普通问题实时解答，疑难问题24小时之内回复。
5.请打开压缩包中的论文Word文档查看批注的错误信息并进行修改。

检测结论

问题总数：5　万字差错率：1.62/10000　结论：合格。请继续修改剩下的错误

评阅教师：066robot　2022-05-10 20:43:49　用时：01:02